MATH 345 Skeleton Notes
Unit 4: Orthogonality and least squares
Section 4.4: QR factorization and least squares

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

From Gram–Schmidt to QR

The Gram-Schmidt algorithm processes the independent columns of a matrix from left to right. It constructs orthonormal vectors $\mathbf{q}_1, \dots, \mathbf{q}_n$ and records coefficients r_{ij}. These same quantities form a matrix factorization.

Let

$$A = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_n \end{bmatrix}.$$

At column j, the Gram–Schmidt algorithm computes

$$r_{ij} = \mathbf{q}_i^T \mathbf{a}_j, \qquad i < j,$$

$$\mathbf{p}_j = \sum_{i<j} r_{ij}\mathbf{q}_i, \qquad \mathbf{v}_j = \mathbf{a}_j - \mathbf{p}_j,$$

$$r_{jj} = \|\mathbf{v}_j\|, \qquad \mathbf{q}_j = \frac{\mathbf{v}_j}{r_{jj}}.$$

Since

$$\mathbf{v}_j = r_{jj}\mathbf{q}_j,$$

we can reconstruct the original column:

$$\mathbf{a}_j = \mathbf{p}_j + \mathbf{v}_j = \sum_{i<j} r_{ij}\mathbf{q}_i + r_{jj}\mathbf{q}_j = \sum_{i\le j} r_{ij}\mathbf{q}_i.$$

Thus column j uses only $\mathbf{q}_1, \dots, \mathbf{q}_j$.
Collect the orthonormal vectors in

$$Q = \begin{bmatrix} \mathbf{q}_1 & \mathbf{q}_2 & \cdots & \mathbf{q}_n \end{bmatrix}$$

and the coefficients in

$$R = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & r_{nn} \end{bmatrix}.$$

The column equations above become the single matrix equation

$$A = QR.$$

The zeros below the diagonal of R record the order of the algorithm: column j uses only directions constructed by stage j.

Definition: Reduced QR factorization

Let A be an $m \times n$ matrix with

linearly independent columns.

A reduced QR factorization of A is a factorization

$$A = QR,$$

where

- Q is an $m \times n$ matrix with orthonormal columns;

- R is an invertible upper-triangular $n \times n$ matrix.

For the Gram–Schmidt convention used in this unit, the diagonal entries of R are positive:

$$r_{jj} = \|\mathbf{v}_j\| > 0.$$

Activity: Reading Q^T Q (U4-LO1)

Let

$$Q = \begin{bmatrix} 1/\sqrt{3} & 1/\sqrt{6} \\ 1/\sqrt{3} & -2/\sqrt{6} \\ 1/\sqrt{3} & 1/\sqrt{6} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1 & \mathbf{q}_2 \end{bmatrix}.$$

1. Compute $\|\mathbf{q}_1\|^2$ and $\|\mathbf{q}_2\|^2$.
2. Compute $\mathbf{q}_1^T \mathbf{q}_2$.
3. Compute $Q^T Q$.
4. What does the (i, j)-entry of $Q^T Q$ measure?
5. Explain why

$$Q^T Q = I_2$$

is the matrix check that the columns of Q are orthonormal.

Theorem: Gram–Schmidt produces QR

Every $m \times n$ matrix A with

linearly independent columns has a reduced QR factorization

$$A = QR$$

in which the columns of Q are orthonormal and R is upper triangular with positive diagonal entries.

Why is this true?

Apply the Gram-Schmidt algorithm to the columns

$$\mathbf{a}_1, \ldots, \mathbf{a}_n.$$

At stage j, the algorithm gives

$$\mathbf{a}_j = \sum_{i \leq j} r_{ij} \mathbf{q}_i.$$

Collecting these equations column by column gives

$$A = QR.$$

The columns of Q are orthonormal. The condition $i \leq j$ makes R upper triangular, and

$$r_{jj} = \|\mathbf{v}_j\| > 0$$

because the input columns are linearly independent.

Corollary: Square case

Every invertible square matrix has a QR factorization

$$A = QR,$$

where Q is an orthogonal matrix and R is upper triangular with positive diagonal entries.

Why is this true?

An invertible square matrix has linearly independent columns. Apply "Gram–Schmidt produces QR." Since Q is square and has orthonormal columns,

$$Q^TQ = QQ^T = I.$$

Thus Q is an orthogonal matrix.

Activity: Build Q and R from the Gram–Schmidt ledger (U4-LO2, U4-LO6)

Let

$$A = \begin{bmatrix} 3 & 1 \\ 4 & 2 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 \end{bmatrix}.$$

1. Compute

$$r_{11} = \|\mathbf{a}_1\|$$

and

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{r_{11}}.$$

2. Compute the projection coefficient

$$r_{12} = \mathbf{q}_1^T\mathbf{a}_2.$$

3. Compute

$$\mathbf{p}_2 = r_{12}\mathbf{q}_1$$

and

$$\mathbf{v}_2 = \mathbf{a}_2 - \mathbf{p}_2.$$

Activity: Build Q and R from the Gram–Schmidt ledger (U4-LO2, U4-LO6) (continued)

4. Compute

$$r_{22} = \|\mathbf{v}_2\|$$

and

$$\mathbf{q}_2 = \frac{\mathbf{v}_2}{r_{22}}.$$

5. Assemble

$$Q = \begin{bmatrix} \mathbf{q}_1 & \mathbf{q}_2 \end{bmatrix}$$

and

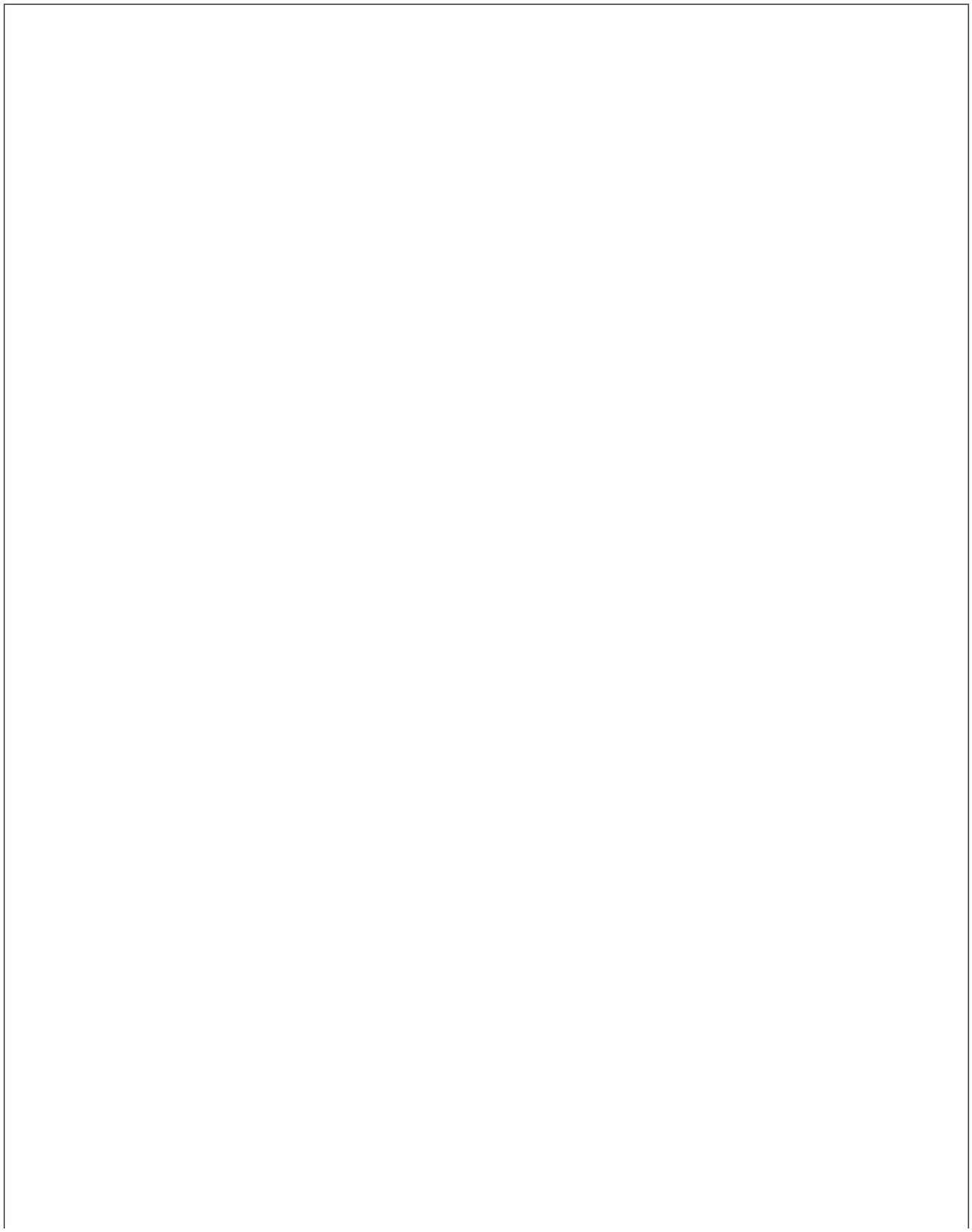
$$R = \begin{bmatrix} r_{11} & r_{12} \\ 0 & r_{22} \end{bmatrix}.$$

6. Check

$$Q^T Q = I_2$$

and

$$QR = A.$$



Activity: An upper-triangular matrix already has a QR factorization (U4-LO6)

Let

$$A = \begin{bmatrix} 6 & 2 \\ 0 & 3 \end{bmatrix}.$$

Explain why

$$Q = I_2, \qquad R = A$$

Activity: An upper-triangular matrix already has a QR factorization (U4-LO6) (continued)

is a QR factorization. Check each condition in the definition.

Warning: Rectangular Q

In a reduced QR factorization of an $m \times n$ matrix with $m > n$, the matrix Q has orthonormal columns, so

$$Q^T Q = I_n.$$

However, Q is not square, and generally

$$QQ^T \neq I_m.$$

Instead,

$$QQ^T\mathbf{b} = \text{proj}_{\text{col}(Q)}(\mathbf{b}) = \text{proj}_{\text{col}(A)}(\mathbf{b}).$$

Indeed, the entries of $Q^T\mathbf{b}$ are the dot products $\mathbf{q}_i \cdot \mathbf{b}$. Multiplying by Q forms the corresponding linear combination of its columns:

$$QQ^T\mathbf{b} = \sum_{i=1}^{n}(\mathbf{q}_i \cdot \mathbf{b})\mathbf{q}_i = \text{proj}_{\text{col}(Q)}(\mathbf{b}).$$

The last equality follows from the Projection theorem, since the columns of Q form an orthonormal basis of $\text{col}(Q)$.
Also, $A = QR$ expresses every column of A as a linear combination of the columns of Q. Since R is invertible, $Q = AR^{-1}$ gives the reverse inclusion. Thus $\text{col}(Q) = \text{col}(A)$, so the two projections agree.

Warning: Signs in a QR factorization

The positive-diagonal convention fixes the signs of the Gram–Schmidt factorization. Without that convention, one may replace a column $\mathbf{q}_j$ by $-\mathbf{q}_j$ and replace row j of R by its negative. The product QR does not change. Numerical software may therefore return Q and R with signs different from a hand calculation. Check $Q^TQ = I$ and $QR = A$ rather than requiring every entry to have the same sign.

Least squares with QR

The normal equations in Least squares and the normal equations give a direct route from a QR factorization to a least-squares solution.
Suppose

$$A = QR$$

is a reduced QR factorization. The normal equations are

$$A^T A\hat{\mathbf{x}} = A^T \mathbf{b}.$$

Substituting $A = QR$ gives

$$R^T Q^T QR\hat{\mathbf{x}} = R^T Q^T \mathbf{b}.$$

Since

$$Q^T Q = I,$$

this becomes

$$R^T R\hat{\mathbf{x}} = R^T Q^T \mathbf{b}.$$

The matrix R is invertible, so R^T is invertible. Canceling R^T gives

$$R\hat{\mathbf{x}} = Q^T \mathbf{b}.$$

The next activity shows why this upper-triangular system is easy to solve by back-substitution, the method introduced in Solving a system using back substitution (U2-LO1, U2-LO2) in Unit 2.

Activity: Back-substitution (U2-LO2)

Solve

$$R\mathbf{x} = \mathbf{y},$$

where

$$R = \begin{bmatrix} 4 & 2 & 7 \\ 0 & 3 & 5 \\ 0 & 0 & 2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 29 \\ 21 \\ 6 \end{bmatrix}.$$

Work from the last equation upward.

Algorithm: Least squares with QR

Input. An $m \times n$ matrix A with linearly independent columns and a vector $\mathbf{b} \in \mathbb{R}^m$.

1. Compute a reduced QR factorization

$$A = QR.$$

2. Compute

$$\mathbf{y} = Q^T \mathbf{b}.$$

3. Solve the upper-triangular system

$$R\hat{\mathbf{x}} = \mathbf{y}$$

by back-substitution.

Output. The vector $\hat{\mathbf{x}}$ is the least-squares coefficient vector. The fitted vector is

$$A\hat{\mathbf{x}} = QR\hat{\mathbf{x}} = QQ^T \mathbf{b}.$$

This method avoids forming $A^T A$ or computing a matrix inverse: once Q and R are known, we need only dot products and back-substitution. If A stays fixed while $\mathbf{b}$ changes, we can reuse the same factorization for each new right-hand side. In floating-point computation, a numerically stable QR routine can also be more accurate than solving the normal equations, especially when the columns of A are nearly linearly dependent.

Activity: Solving least squares with QR (U4-LO2, U4-LO6)

Use QR to fit a line $y = c_0 + c_1 t$ to the data points

$$(0, 1), \qquad (1, 2), \qquad (2, 2)$$

from Fit a line to three data points (U4-LO4, U4-LO5). The design matrix and data vector are

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

1. Apply Gram–Schmidt to the columns of X to find a reduced factorization $X = QR$ with positive diagonal entries in R.
2. Compute $Q^T \mathbf{y}$.
3. Solve $R\hat{\mathbf{c}} = Q^T \mathbf{y}$ by back-substitution and write the fitted line.

Activity: Solving least squares with QR by code (U4-LO6, U4-LO7)

The following code repeats the QR least-squares calculation from Solving least squares with QR (U4-LO2, U4-LO6).
The function `np.linalg.qr` returns the two factors, which the

Activity: Solving least squares with QR by code (U4-LO6, U4-LO7) (continued)

assignment stores in `Q` and `R`: `Q` has orthonormal columns, `R` is upper triangular, and their product is `X`. The option `mode="reduced"` keeps only the columns of `Q` needed to span the column space of `X`, together with the corresponding square factor `R`, rather than extending `Q` to a square orthogonal matrix.

The expression `np.eye(Q.shape[1])` constructs an identity matrix whose size is the number of columns of `Q`; here it is the 2×2 identity matrix.

The function `np.linalg.solve` solves a square system directly, without explicitly computing the inverse of its coefficient matrix. Using it is preferable to forming an inverse and then multiplying, which adds unnecessary computation and can introduce additional rounding error.

```
import numpy as np

t = np.array([0.0, 1.0, 2.0])
y = np.array([1.0, 2.0, 2.0])

X = np.column_stack([np.ones_like(t), t])

Q, R = np.linalg.qr(X, mode="reduced")

c_qr = np.linalg.solve(R, Q.T @ y)
c_lstsq = np.linalg.lstsq(X, y, rcond=None)[0]

(
    Q.shape,
    R.shape,
    np.allclose(Q.T @ Q, np.eye(Q.shape[1])),
    np.allclose(Q @ R, X),
    c_qr,
    c_lstsq,
    np.allclose(c_qr, c_lstsq),
```

Activity: Solving least squares with QR by code (U4-LO6, U4-LO7) (continued)

```
)
```

1. What should `Q.shape` and `R.shape` be?
2. What does Q.T @ Q
   check?
3. What does Q @ R
   check?
4. What mathematical vector is computed by Q.T @ y
   ?
5. Why does the code solve a system with `R`?
6. Why should `c_qr` and `c_lstsq` agree?
7. Does their agreement mean that

$$X\mathbf{c} = \mathbf{y}$$

   has an exact solution?
8. Why may the entries of `Q` and `R` have signs different from a hand Gram–Schmidt calculation?